



Wydział Technologii Informatycznych

Kierunek studiów: Informatyka

Forma studiów: studia stacjonarne

Vadym Abrosimov
Nr albumu 43995

System detekcji dezinformacji w mediach społecznościowych

Dokumentacja projektu informatycznego
przygotowana pod kierunkiem
dr. inż. Andrzeja Burdy

Warszawa, 2026

1. Podstawowe informacje

Nazwa projektu

System detekcji dezinformacji i błędów logicznych w tekstach medialnych z wykorzystaniem wyjaśnialnej sztucznej inteligencji (XAI)

Cel projektu

Cel główny: Celem projektu jest opracowanie i wdrożenie systemu informatycznego służącego do automatycznej identyfikacji oraz wyjaśniania technik manipulacji i błędów logicznych w polskojęzycznych artykułach. Kluczowym aspektem jest nie tylko klasyfikacja, ale również generowanie zrozumiałego dla użytkownika uzasadnienia (tzw. łańcuch rozumowania – Chain-of-Thought), co odróżnia ten system od klasycznych klasyfikatorów typu "czarna skrzynka".

Rozwiązywane problemy: Projekt adresuje luki w istniejących rozwiązaniach dwójako:

Dla użytkownika końcowego: Rozwiązuje problem "czarnej skrzynki" w automatycznej weryfikacji treści. Zamiast binarnej etykiety "Fałsz", system generuje zrozumiałe uzasadnienie, pełniące funkcję wyjasniającą i zwiększające kompetencje medialne odbiorcy.

Dla inżyniera/analityka: Rozwiązuje problem degradacji modeli w czasie. Dzięki modułowi z udziałem człowieka (Human-in-the-Loop), system umożliwia ekspertowi natychmiastowe douczanie modelu (metodą LoRA). Pozwala to na szybką adaptację narzędzia do nowych narracji dezinformacyjnych, z którymi statyczne modele sobie nie radzą.

Krótki opis projektu

Projekt stanowi kompleksowe rozwiązanie inżynierskie integrujące przetwarzanie języka naturalnego (NLP) z interaktywną aplikacją webową. Rdzeniem systemu jest model językowy (speakeash/Bielik-4.5B-v3.0-Instruct), który został nauczony nie tylko klasyfikacji, ale i argumentacji.

Metodyka i Architektura: Kluczowym elementem prac jest przygotowanie wysokiej jakości zbioru danych treningowych w oparciu o zbiór MIPD. Zastosowano strategię "**Label-Conditional Distillation**" (**destylację warunkową pod etykietą**), gdzie większy model nauczycielski (unsloth/Qwen2.5-7B-Instruct-bnb-4bit) generuje uzasadnienia logiczne ściśle dopasowane do decyzji ludzkich ekspertów. Pozwala to na zachowanie "ludzkiej" oceny, przy jednoczesnym uzyskaniu spójnych wyjaśnień, na których uczony jest mniejszy model.

Aplikacja składa się z dwóch interfejsów:

Panel Użytkownika: Umożliwia wklejenie tekstu i otrzymanie analizy (wykryte techniki manipulacji + wyjaśnienie).

Panel Inżynierski: Pozwala na ładowanie nowych danych treningowych oraz uruchomienie procesu **lokalnego dotrenowania**. Wykorzystanie biblioteki Unsloth oraz techniki kwantyzacji pozwala na realizację tego procesu na sprzęcie klasy konsumenckiej, czyniąc system samowystarczalnym i tanim w eksploatacji.

Tabela 1. Klasy manipulacji, które wykrywa system.

Tag	Opis
REFERENCE_ERROR	Powoływanie się na niewiarygodne lub fałszywe źródła.
WHATABOUTISM	Odchodzenie od tematu poprzez wprowadzenie niezwiązanego argumentu.
STRAWMAN	Przeinaczanie argumentu w celu ułatwienia ataku.

EMOTIONAL_CONTENT	Manipulowanie emocjami czytelnika w celu zmiany opinii.
CHERRY_PICKING	Selektywne prezentowanie danych w celu poparcia konkretnego argumentu.
FALSE_CAUSE	Zakładanie związku przyczynowo-skutkowego na podstawie samej korelacji.
MISLEADING_CLICKBAIT	Tworzenie mylącego lub sensacyjnego nagłówka, który jest sprzeczny z treścią artykułu.
ANECDOTE	Wykorzystywanie osobistych historii do podważania danych statystycznych.
LEADING_QUESTIONS	Stawianie pytań w sposób sugerujący z góry ustalony wniosek.
EXAGGERATION	Wyołbrzymianie lub niedoceniające faktów.
QUOTE_MINING	Zniekształcanie czyichś wypowiedzi poprzez wykorzystywanie wybranych fragmentów.

Analiza konkurencji

Rynek rozwiązań służących do detekcji dezinformacji jest obecnie podzielony między gigantów technologicznych a specjalistyczne, lecz ograniczone funkcjonalnie projekty badawcze. Niniejszy projekt pozycjonuje się jako rozwiązanie hybrydowe, łączące zalety obu podejść przy jednoczesnym wyeliminowaniu ich kluczowych wad.

Pierwszym punktem odniesienia są **komercyjne modele językowe ogólnego przeznaczenia**, takie jak GPT-4 czy Gemini. Choć oferują one wysoki poziom wnioskowania, ich stosowanie w systemach bezpieczeństwa informacyjnego obarczone jest poważnymi ryzykami. Są to rozwiązania typu "zamkniętego" (API), co rodzi problemy z prywatnością analizowanych danych oraz kosztami eksploatacji. Co istotne, modele te posiadają wbudowane restrykcyjne filtry bezpieczeństwa (tzw. over-refusal), przez co często odmawiają analizy kontrowersyjnych tekstów politycznych, które są głównym nośnikiem dezinformacji. Ponadto, jako modele globalne, często niepoprawnie interpretują lokalny kontekst kulturowy (np. polską ironię polityczną), traktując ją dosłownie.

Drugą grupę stanowią **klasyczne modele klasyfikacyjne oparte na architekturze BERT** (np. PL-RoBERTa), powszechnie stosowane w polskiej nauce. Ich główną zaletą jest szybkość i niski koszt obliczeniowy. Są to jednak systemy "niewyjaśnialne" – zwracają jedynie wynik liczbowy (prawdopodobieństwo), nie tłumacząc użytkownikowi, dlaczego tekst został uznany za manipulację. Ogranicza to zaufanie do narzędzia i jego walor edukacyjny. Dodatkowo modele te są statyczne; ich aktualizacja wymaga specjalistycznej wiedzy programistycznej i pełnego cyklu treningowego, co uniemożliwia szybką reakcję na nowe trendy w dezinformacji.

Trzecią kategorią są **portale fact-checkingowe** oparte na pracy ludzkiej (np. Demagog). Stanowią one "złoty standard" wiarygodności, jednak są całkowicie nieskalowalne. Czas weryfikacji jednego artykułu liczy się w godzinach, co sprawia, że nie są w stanie monitorować strumienia treści w czasie rzeczywistym.

Przewaga proponowanego rozwiązania opiera się na trzech filarach. Po pierwsze, w przeciwieństwie do modeli BERT, system oferuje pełną **wyjaśnialność decyzji** (uzasadnianie predykcji – Reasoning), budując zaufanie użytkownika. Po drugie, w odróżnieniu od GPT-4, jest to model **lokalny i uncenzurowany**, zdolny do analizy dowolnych treści bez ograniczeń

korporacyjnych i kosztów API. Po trzecie, unikalny **moduł inżynierski ("Retrain Button")** wprowadza funkcjonalność ciągłego uczenia się, pozwalając operatorowi na adaptację modelu do nowych narracji w kilkanaście minut, co stanowi istotną innowację w stosunku do statycznych rozwiązań akademickich.

Wykaz zastosowanych technologii

- Modele Językowe (LLM): Bielik-4.5B-Instruct, Qwen-2.5-7B-Instruct.
- Biblioteki i Frameworki ML: Unsloth, Hugging Face Transformers, Hugging Face PEFT.
- Serwery i Backend: Ollama (oparty na llama.cpp), Python (FastAPI).
- Frontend: React.js.
- Sprzęt i Środowisko: Google Colab (T4 GPU), lokalna stacja robocza (GPU NVIDIA).

Opis stosu technologicznego i uzasadnienie wybranych technologii

Modele Językowe (Bielik-4.5B & Qwen-2.5-7B):

Rola: Serce systemu. Bielik-4.5B służy jako model produkcyjny (inferencja), a Qwen-2.5-7B jako model "nauczyciel" do generowania danych syntetycznych.

Uzasadnienie: Bielik jest modelem specjalnie dostrojonym do języka polskiego, co jest kluczowe dla detekcji niuansów językowych w polskich mediach. Jego rozmiar (4.5B) pozwala na uruchomienie na standardowym sprzęcie konsumenckim. Qwen 7B, będąc modelem większym i bardziej ogólnym, posiada lepsze zdolności logicznego wnioskowania, co czyni go idealnym do generowania uzasadnień (tzw. łańcuch rozumowania) w fazie treningu.

Unsloth (Trening i Optymalizacja):

Rola: Biblioteka do fine-tuningu modelu Bielik.

Uzasadnienie: Unsloth umożliwia trenowanie modeli LLM nawet 2x szybciej i przy zużyciu o 60% mniej pamięci VRAM niż standardowe metody. Pozwala to na przeprowadzenie procesu SFT (nadzorowane dostrajanie – Supervised Fine-Tuning) na darmowej instancji Google Colab (T4 GPU), co czyni projekt wykonalnym bez dostępu do klastrów obliczeniowych.

Ollama (Serwer Inferencji):

Rola: Lokalny serwer udostępniający model poprzez REST API.

Uzasadnienie: Ollama automatyzuje zarządzanie modelem, obsługuje wydajny format GGUF (kwantyzacja) i udostępnia proste API kompatybilne z aplikacjami webowymi. Eliminuje konieczność pisania skomplikowanego kodu w Pythonie do obsługi ładowania modelu i tokenizacji w środowisku produkcyjnym.

Python (FastAPI) & React.js:

Rola: Orkiestracja procesów (Backend) i interfejs użytkownika (Frontend).

Uzasadnienie: FastAPI jest nowoczesnym, asynchronicznym frameworkiem idealnym do obsługi długotrwałych zadań w tle (jak trenowanie modelu w fazie 4). React.js zapewnia responsywny interfejs, który może dynamicznie wyświetlać strumieniowane odpowiedzi z modelu (efekt pisania na żywo).

Google Colab (T4 GPU):

Rola: Środowisko treningowe.

Uzasadnienie: Zapewnia darmowy dostęp do akceleratora GPU niezbędnego do operacji na tensorach i trenowania sieci neuronowych, co jest kluczowe w fazie badawczej projektu.

2. Kluczowe zagadnienia związane z realizacją projektu

Realizacja systemu detekcji dezinformacji wymagała rozwiązania szeregu problemów inżynierskich z zakresu przetwarzania języka naturalnego (NLP), optymalizacji modeli głębokiego uczenia oraz architektury systemów rozproszonych. Poniżej przedstawiono

kluczowe aspekty implementacyjne z podziałem na etapy wytwarzania oprogramowania, uwzględniające rzeczywiste parametry i wyniki uzyskane w toku prac.

2.1. Cele i charakterystyka inżynierska projektu

Zasadniczym celem pracy było zaprojektowanie i implementacja kompletnego systemu informatycznego zdolnego do wykrywania technik manipulacji w tekstach polskojęzycznych wraz z generowaniem zrozumiałych dla człowieka wyjaśnień decyzji (Explainable AI). Projekt koncentrował się na stworzeniu działającego artefaktu inżynierskiego, który integruje nowoczesne modele językowe w spójną architekturę aplikacyjną.

Co było celem:

Stworzenie działającego prototypu systemu MLOps, który umożliwia nie tylko inferencję, ale także ciągłe douczanie modelu z udziałem człowieka (tzw. Human-in-the-Loop) w oparciu o interakcję z ekspertem. Kluczowe było rozwiązanie problemów integracyjnych (backend, frontend, obsługa GPU) oraz optymalizacja modelu 4.5B parametrów do działania na sprzęcie konsumenckim.

Co nie było celem:

Celem pracy nie było przeprowadzanie wyczerpujących badań porównawczych wielu architektur sieci neuronowych ani tworzenie nowych architektur modeli fundamentalnych. Nie dążono również do osiągnięcia wyników stanu wiedzy (State-of-the-Art, SOTA) w rozumieniu akademickim za wszelką cenę, lecz do uzyskania kompromisu między jakością detekcji a użytecznością i szybkością działania systemu lokalnego.

Uzasadnienie spełnienia kryteriów pracy inżynierskiej:

Projekt wykracza poza teoretyczną analizę problemu, dostarczając w pełni funkcjonalne oprogramowanie. Wymagał on doboru odpowiednich narzędzi (Unsloth, Ollama, FastAPI), zaprojektowania bazy danych, implementacji algorytmów przetwarzania tekstu oraz stworzenia interfejsu użytkownika. Stanowi więc klasyczny przykład inżynierii oprogramowania połączonej z inżynierią danych.

2.2. Implementacja procesu generowania danych syntetycznych (NLE i generowanie uzasadnień – Rationale Generation)

Jednym z największych wyzwań projektu był brak zbioru danych zawierającego nie tylko etykiety (np. "REFERENCE_ERROR"), ale również wyjaśnienia w języku naturalnym (Natural Language Explanations - NLE). Koncepcja NLE, wprowadzona m.in. przez Camburu et al. (2018), zakłada generowanie tekstowego uzasadnienia decyzji klasyfikatora, co jest kluczowe dla budowania zaufania użytkownika w systemach Explainable AI (XAI). Podwaliny pod to podejście położyły również prace takie jak Lei et al. (2016), koncentrujące się na ekstrakcji fragmentów tekstu uzasadniających predykcję (Rationale Generation). Ścisłe powiązanie klasyfikacji z generowaniem uzasadnień zapobiega tworzeniu modeli typu "czarna skrzynka".

Wyzwanie implementacyjne:

Konieczność wygenerowania tysięcy wysokiej jakości uzasadnień dla zbioru MIPD (Modzelewski et al., 2024) (ponad 10 000 próbek) przy konieczności zachowania spójności

formatu JSON. Istotnym aspektem było tzw. generowanie uzasadnień (Lei et al., 2016), czyli proces ekstrakcji fragmentów tekstu źródłowego, które bezpośrednio uzasadniają predykcję.

Zastosowane rozwiązanie:

Zaimplementowano autorski potok (pipeline) generacji metodologią "Nauczyciel-Uczeń" (Teacher-Student; Hsieh et al., 2023) z wykorzystaniem modelu Qwen-2.5-7B-Instruct (Yang et al., 2024) jako "Nauczyciela". Proces ten zrealizowano w środowisku chmurowym Google Colab z wykorzystaniem biblioteki Unsloth (Han & Liu, 2023) dla optymalizacji pamięci VRAM (model 4-bitowy). Wybór modelu Nauczyciela o parametrach 7B podyktowany był potrzebą zapewnienia wysokiej jakości merytorycznej generowanych wyjaśnień oraz ścisłego przestrzegania schematu JSON, co w przypadku mniejszych modeli stanowiłoby ryzyko niestabilności formatu.

Kluczowym elementem implementacji był skrypt Python (`synthetic_data_gen.ipynb`), który wymuszał na modelu przestrzeganie reguł ścisłego generowania (Hard-Constraint Generation):

Iteracyjna walidacja (pętla ponawiania – Retry Loop): Zaimplementowano pętlę ponawiania prób. Jeśli model wygenerował odpowiedź niepoprawną składniowo (nie zawierającą techniki manipulacji), system automatycznie ponawiał zapytanie.

Mechanizm wznowiania (logika wznowiania – Resume Logic): Ze względu na czasochłonność procesu, zaimplementowano system punktów kontrolnych (checkpoints), zapobiegający utracie danych w przypadku rozłączenia sesji Colab.

Separacja logiki: Model generował treść wyjaśnienia, a struktura JSON była składana programowo, co eliminowało błędy składniowe. W docelowym systemie (XAI Adapter) model jest trenowany, aby na wyjściu zwracać spójną strukturę zawierającą listę zidentyfikowanych technik oraz pole uzasadnienia:

```
{
  "reasoning": "Na podstawie analizy podanych fragmentów, tekst wykazuje cechy selektywnego doboru faktów, co w połączeniu z wyolbrzymieniem skali zjawiska służy budowaniu narracji...",
  "discovered_techniques": ["CHERRY_PICKING", "EXAGGERATION"]
}
```

2.3. Metodyka dostrajania modelu (nadzorowane dostrajanie – Supervised Fine-Tuning)

Sercem systemu jest model Bielik-4.5B-Instruct (SpeakLeash Team, 2024). Bezpośrednie użycie modelu bazowego dawało wyniki niesatysfakcjonujące w kontekście specyficznego zadania detekcji manipulacji, co wymusiło proces dostrajania (Fine-Tuning).

Wyzwanie implementacyjne:

Standardowy trening modelu o 4.5 miliarda parametrów wymagał optymalizacji pamięciowej oraz obsługi długich tekstów (artykuły z datasetu MIPD często przekraczają standardowe okna kontekstowe).

Zastosowane rozwiązanie:

Zastosowano technikę QLoRA (Dettmers et al., 2023) z wykorzystaniem biblioteki Unsloth. Pozwoliło to na zamrożenie głównych wag modelu (zapisanych w formacie 4-bitowym NF4) i trenowanie jedynie niewielkich macierzy adapterów (Hu et al., 2021). Proces konfiguracji warstwy adaptacyjnej (LoRA) przedstawia poniższy fragment kodu (plik `trainer.py`):

```
model = FastLanguageModel.get_peft_model(
    model,
    r = 16, # Ranga macierzy LoRA
    target_modules = ["q_proj", "k_proj", "v_proj", "o_proj", "gate_proj",
"up_proj", "down_proj"],
    lora_alpha = 16,
    lora_dropout = 0,
    bias = "none",
    use_gradient_checkpointing = "unsloth",
)
```

W celu obsługi długich tekstów zaimplementowano strategię Long Context z wykorzystaniem RoPE Scaling (Liu et al., 2023). W konfiguracji trenera ustawiono parametr `max_seq_length = 16384`, co w połączeniu z `gradient_checkpointing` umożliwiło trening na dostępnych zasobach (w Colabie). Dodatkowo, zastosowano technikę maskowanie promptu (tzw. Completion Only LM) (maskowanie promptu użytkownika w funkcji straty), co zmusiło model do uczenia się wyłącznie generowania analizy, a nie reprodukcji długiego tekstu wejściowego, znacząco przyspieszając konwergencję. Model prototypowy osiągnął wynik F1 na poziomie 0.7824 na zbiorze testowym, dowodząc technicznej wykonalności przyjętej koncepcji.

2.4. Architektura inferencji i konwersja modelu

Kluczowym wymaganiem projektowym była optymalizacja procesu inferencji, aby umożliwić działanie modelu w środowisku o ograniczonych zasobach, przy jednoczesnym zachowaniu akceptowalnego czasu odpowiedzi.

Wyzwanie implementacyjne:

Uruchomienie modelu o długim kontekście (16k tokenów) z zachowaniem interaktywności i precyzji odpowiedzi ustrukturyzowanych (JSON).

Zastosowane rozwiązanie:

Zastosowano architekturę dynamicznego ładowania adapterów (Runtime Adapter Loading) w serwerze Ollama. Podejście to polega na dynamicznym ładowaniu adaptera LoRA na wysokiej jakości model bazowy (Bielik-4.5B w kwantyzacji 8-bitowej). W pliku konfiguracyjnym Modelfile ustawiono temperaturę generacji na 0.1 oraz zdefiniowano precyzyjny szablon ChatML, co jest kluczowe dla determinizmu zwracanych struktur danych.

2.5. Implementacja systemu automatycznej ewaluacji (Auto-Benchmark)

W projektach opartych na LLM, gdzie wyjście jest tekstem strukturalnym (JSON), standardowe metryki (jak accuracy) są niewystarczające. Zaimplementowano autorski system ewaluacji (`benchmark.py`), który analizuje odpowiedzi modelu na wydzielonym zbiorze testowym w trzech wymiarach:

1. Wskaźnik Sukcesu Parsowania (Parsing Success Rate - PSR): Techniczna metryka weryfikująca, czy odpowiedź modelu jest poprawnym obiektem JSON. Skrypt podejmuje dwie próby deserializacji: ścisłą (strict JSON) oraz opartą na wyrażeniach regularnych (regex recovery), aby odzyskać strukturę nawet w przypadku drobnych błędów formatowania (np. brak klamry zamykającej) lub literówek w kluczach (np. "discovered_tech_niques"). System inferencyjny (backend) implementuje ten sam mechanizm "Heurystycznego Odzyskiwania" (Heuristic Recovery), który normalizuje nazwy kluczy oraz naprawia typowe literówki w wartościach tagów (np. "CHERRY_PlckING" -> "CHERRY_PICKING") przed zwróceniem wyniku do użytkownika. Wysoki PSR (>95%) jest warunkiem koniecznym do wdrożenia.
2. Zgodność z etykietami (Exact-Match Accuracy): Odsetek dokumentów, w których model idealnie odtworzył zbiór technik (zbiór predykcji jest identyczny ze zbiorem referencyjnym). Jest to bardzo rygorystyczna miara, karząca za każdą nadmiarową lub brakującą etykietę.
3. Wydajność Klasyfikacji (Mean Document-Level F1 - excluding empty gold-label docs): Główna metryka decyzyjna. Obliczana jako średnia harmoniczna precyzji i czułości (F1) liczona niezależnie dla każdego dokumentu, ale tylko dla tych próbek, które w rzeczywistości zawierają techniki manipulacji (niepuste gold labels).

* Uzasadnienie: Ze względu na niezbalansowanie zbioru danych (duża liczba "czystych" artykułów), wliczanie pustych przykładów sztucznie zawyżałoby wynik (model łatwo uczy się przewidywać pustą listę). Skupienie się na niepustych przykładach pozwala ocenić rzeczywistą zdolność modelu do detekcji manipulacji, a nie tylko jego tendencję do bycia konserwatywnym.

System ewaluacji jest w pełni zautomatyzowany i zintegrowany z backendem – po każdym cyklu douczania uruchamiany jest na losowej próbce danych testowych, a wynik F1 decyduje o promocji modelu na produkcję.

2.6. Projekt i architektura Systemu Inżynierskiego (MLOps Loop)

Istotą projektu jako pracy inżynierskiej jest transformacja statycznego modelu w adaptujący się system. Zaprojektowano architekturę MLOps z pętlą zwrotną z udziałem człowieka (Human-in-the-Loop), składającą się z:

1. Backendu Orkiestracyjnego: Serwisu w Pythonie, który monitoruje przyrost danych i zarządza procesami w tle. Wykorzystuje on środowisko WSL (Windows Subsystem for Linux) jako warstwę izolacji obliczeniowej ('orchestrator.py'):

```
# Inicjalizacja treningu w wyizolowanym kontenerze WSL
cmd = f"wsl --exec python3 -u -m app.training.trainer --data {wsl_path} --\noutput ./model/latest --base {base_model_wsl}"
process = subprocess.Popen(cmd, shell=True, stdout=f_log,\nstderr=subprocess.STDOUT)
```

2. Automatyzacji Treningu (wyzwalanie i douczanie – Trigger & Retraining): Mechanizmu, który po zebraniu kompletnego zestawu ewaluacyjnego uruchamia proces douczania.
3. Decyzji o Wdrożeniu: Logiki, która po zakończeniu automatycznego benchmarku zwraca wyniki do eksperta, zapalając odpowiednie wskaźniki w interfejsie graficznym. Interfejs podświetla na zielono lub czerwono kluczowe metryki, ułatwiając podjęcie decyzji. Ekspert zachowuje jednak pełną kontrolę – może zatwierdzić wdrożenie – dynamiczną wymianę

modelu (tzw. Hot-Swap) na produkcję poprzez kliknięcie przycisku "Wdroż", nawet jeśli wyniki numeryczne uległy pogorszeniu, jeśli np. uzna, że nowa wersja lepiej radzi sobie z najcięższymi przypadkami brzegowymi.

Algorytm procesu iteracyjnego douczania (pętla douczania – Retraining Loop):

1. Odbierz nowy, kompletny zbiór danych treningowych przesłany przez eksperta przez interfejs.
2. Inicjuj proces treningowy (adapter LoRA) we wskazanym środowisku wykonawczym (WSL).
3. Po zakończeniu, uruchom automatyczną ewaluację modelu (Benchmark) na wydzielonym zbiorze testowym.
4. Pobierz wyniki ewaluacji (F1 Score, Exact Match).
5. Wyświetl wyniki w panelu wraz z kolorystyczną sugestią wdrożenia (czerwony/zielony).
6. Oczekuj na manualną akceptację przez eksperta (przycisk "Wdroż" staje się aktywny).
7. W przypadku zatwierdzenia, skompiluj adapter do formatu GGUF (z wykorzystaniem narzędzi llama.cpp; Gerganov, 2023) i podmień aktualny model produkcyjny w serwerze Ollama.

Świadomie zrezygnowano z pełnej automatyzacji wdrożeń, pozostawiając ostateczną decyzję ekspertowi, ponieważ metryki ilościowe nie zawsze odzwierciedlają jakość zachowania modelu w przypadkach brzegowych.

System korzysta z dwuwarstwowego modelu przechowywania danych. Baza SQLite (disinfo_system.db) rejestruje metadane przebiegów treningowych w tabeli TrainingRun (identyfikator, ramy czasowe, wyniki F1 przed i po douczaniu, status, ścieżka adaptera). Warstwa plików systemowych przechowuje: zbiór referencyjny do ewaluacji (model/dataset/mipd_test.jsonl — stały zbiór testowy z adnotacjami gold-label), checkpointy adapterów LoRA (katalog model/latest/) oraz raporty benchmarkowe z podziałem na warianty (model/benchmark-reports/). Gorąca wymiana modelu produkcyjnego polega na konwersji adaptera do formatu GGUF i przeładowaniu serwera Ollama.

2.7. Ograniczenia projektu

Jako realizacja inżynierska, projekt posiada pewne ograniczenia wynikające z przyjętego zakresu prac oraz dostępnych zasobów:

Ograniczenia infrastrukturalne: Ze względu na ograniczone zasoby sprzętowe (brak dostępu do środowiska serwerowego z dużą ilością pamięci VRAM), system został wdrożony i przetestowany w środowisku lokalnym jako weryfikacja koncepcji (Proof-of-Concept).

Uniemożliwiło to pełne testy wydajnościowe z maksymalnym wykorzystaniem okna kontekstowego (16k-32k tokenów) w warunkach produkcyjnego obciążenia.

Niezbalansowanie zbioru danych: Zbiór treningowy charakteryzuje się znaczną nadreprezentacją próbek pustych (brak technik manipulacji), co wpływa na "konserwatywność" modelu – tendencję do nieoznaczania technik w przypadkach niejednoznacznych, aby zminimalizować liczbę fałszywych alarmów.

Uproszczona autoryzacja eksperta: Funkcjonalność logowania eksperta została zaimplementowana w formie uproszczonego przełącznika interfejsu (mock), pomijając zaawansowane mechanizmy uwierzytelniania (RBAC), co byłoby wymagane przy wdrożeniu komercyjnym.

Brak formalnej walidacji jakości wyjaśnień: Oceniono głównie poprawność klasyfikacji (wykrycie tagu). Jakość generowanych wyjaśnień (NLE) nie została poddana rygorystycznej ocenie z udziałem grupy sędziów kompetentnych (human evaluation), co jest standardem w pracach ściśle badawczych.

Zależność od jakości danych syntetycznych: Skuteczność modelu jest bezpośrednio skorelowana z jakością danych wygenerowanych przez model "Nauczyciela". Ewentualne błędy w rozumowaniu modelu Qwen-2.5-7B mogły zostać powielone w procesie destylacji. Ryzyko biasu w procesie destylacji: Istnieje ryzyko, że model przejął pewne uprzedzenia (bias) zawarte w modelu Nauczyciela, co jest typowym zjawiskiem w procesie destylacji wiedzy (Knowledge Distillation).

Halucynacje nazw kluczy JSON: Mimo programowego wymuszania struktury JSON w zbiorze treningowym (model Nauczyciela generował tylko wartości, a klucze były doklejane), model w trakcie inferencji wykazuje znaczącą tendencję do modyfikacji nazw kluczy (np. "reasonng" zamiast "reasoning"). Zjawisko to sugeruje, że model traktuje nazwy kluczy jako część generowanego tekstu, a nie sztywny kontrakt. Wykrycie i eliminacja tego problemu wymagałaby rozszerzenia benchmarku o ścisłą walidację schematu (schema validation) oraz dalszych badań nad mechanizmem uwagi modelu w kontekście syntaktyki JSON.

Halucynacje nazw tagów (wartości): Obserwowano również przypadki generowania błędnych nazw etykiet, będących literówkami nazw poprawnych (np. "EMOTIORAL_CONTENT" zamiast "EMOTIONAL_CONTENT" lub "MISLEADING_CLICKBAI").

Wskazuje to na problem "rozmycia" rzadkich tokenów w procesie kwantyzacji lub niedostateczną liczbę powtórzeń poprawnych etykiet w zbiorze treningowym względem etykiet pustych. Obecny system ewaluacji traktuje takie tagi jako błędne (False Negative dla poprawnej klasy), ale nie klasyfikuje ich jako błędu strukturalnego.

2.8. Analiza wydajności i wyniki eksperymentalne

Złożoność obliczeniowa i parametry czasowe systemu (czas treningu i inferencji) są ściśle uzależnione od wykorzystanej infrastruktury sprzętowej oraz rozmiaru przetwarzanego zbioru. Należy rozróżnić dwa scenariusze użycia. Po pierwsze, ze względu na zapotrzebowanie na zasoby, pełne cykle dostrajania (SFT) i wyczerpująca ewaluacja głównych wariantów modelu (Tabela 1) zostały przeprowadzone w środowisku Google Colab (T4 GPU) na pełnym zbiorze danych. Po drugie, w przypadku lokalnego modułu iteracyjnego douczania (pętla douczania), działającego na stacji roboczej z użyciem zredukowanego zbioru zgłoszeń dezinformacyjnych, szacunkowy czas jednego cyklu douczania (16 kroków) wynosi około 2 minuty. Czas lokalnej inferencji (generowania analizy w czasie rzeczywistym) dla pojedynczego zgłoszenia wynosi od 2 do 5 sekund na konsumenckiej karcie graficznej.

Przeprowadzono analizę porównawczą trzech wariantów modelu:

1. No Adapter: Model bazowy (Bielik-4.5B-Instruct) bez dostrajania.
2. Prototype Adapter: Model dostrojony pod kątem precyzji klasyfikacji (maksymalizacja F1).
3. XAI Adapter: Model dostrojony do generowania zarówno etykiet, jak i uzasadnień decyzji modelu.

Tabela 2. Zestawienie wyników ewaluacji (zbiór testowy N=1521).

Metryka	No Adapter	Prototype Adapter	XAI Adapter
---------	------------	-------------------	-------------

Parsing Success Rate (Strict)	20.05%	99.93%	96.25%
Mean Document-Level F1 (Non-empty docs)	0.0979	0.4912	0.2847
Exact-Match Accuracy	38.07%	72.91%	73.18%

Analiza wyników:

* Wpływ dostrajania: Model bazowy ("No Adapter") wykazuje krytycznie niską zdolność do formowania poprawnego wyjścia JSON (PSR 20%), co dyskwalifikuje go z zastosowań produkcyjnych. Dostrajanie (Adaptory) podnosi stabilność formatu do poziomu >96%.

* Wpływ jakości generowanych wyjaśnień (XAI): Teoretycznie integracja warstwy wyjaśnialności powinna wspierać precyzję klasyfikacji poprzez wymuszenie głębszej analizy tekstu. Obserwowany spadek skuteczności wariantu "XAI" (F1 0.28 vs 0.49) wskazuje jednak na wpływ jakości danych trenujących w zakresie wyjaśnień (NLE). Jak wskazano w sekcji ograniczeń ("Brak formalnej walidacji jakości wyjaśnień"), model "Nauczyciela" mógł generować halucynacje lub błędne uzasadnienia, które model "Ucznia" następnie powielił, co paradoksalnie zaburzyło proces decyzyjny zamiast go wspomóc. Potwierdza to kluczową rolę weryfikacji jakości danych w procesie destylacji wiedzy (Knowledge Distillation).

* Odniesienie do literatury: Uzyskane wyniki dla wariantu *Prototype Adapter* (F1 = 0.49) przewyższają bazowe wyniki raportowane w literaturze dla zbioru MIPD (Modzelewski et al., 2024), gdzie modele PL-RoBERTa-Large osiągały ważne F1 na poziomie ok. 0.47 (± 0.003). Potwierdza to wysoką skuteczność procesu dostrajania modelu Bielik-4.5B w zadaniu czystej klasyfikacji. Warto jednak zauważyć, że wariant *XAI Adapter*, wprowadzający warstwę wyjaśnialności, uzyskuje obecnie wyniki niższe od literatury (F1 = 0.28). Wskazuje to na istotne wyzwanie inżynierskie: wprowadzenie wyjaśnialności (NLE) wiąże się obecnie z kosztem wydajnościowym (tzw. podatkiem od wyjaśnialności – performance tax). Optymalizacja jakości danych syntetycznych oraz procesu destylacji wiedzy, mająca na celu zbliżenie skuteczności wariantu XAI do poziomu Prototype, stanowi jeden z głównych kierunków dalszych prac.

Należy zauważyć, że metryki nie są w pełni równoważne metodologicznie, ponieważ literatura raportuje weighted F1 dla pełnego zbioru, podczas gdy w niniejszej pracy zastosowano Mean Document-Level F1 dla próbek niepustych.

Analiza Macierzy Pomyłek:

Analiza macierzy pomyłek (zawartych w Sekcji 3) potwierdza te obserwacje:

* Adapter Prototype najskuteczniej identyfikuje subtelne techniki jak EXAGGERATION (187 TP) czy CHERRY_PICKING (101 TP).

* Adapter XAI przyjmuje strategię bardziej konserwatywną, częściej "przeocza" techniki (więcej False Negatives), np. dla EXAGGERATION wykrył 128 przypadków.

* Model bez adaptera w większości przypadków nie wykrywa technik (ogromna przewaga False Negatives nad True Positives dla niemal wszystkich klas), co potwierdza konieczność treningu specyficznego dla domeny detekcji dezinformacji.

2.9. Instrukcja instalacji i uruchomienia systemu

Kompletna instrukcja odtworzenia środowiska uruchomieniowego jest kluczowym elementem dokumentacji inżynierskiej. Poniższa sekcja opisuje wymagania sprzętowe i programowe, proces automatycznej konfiguracji oraz kolejność uruchomienia poszczególnych komponentów systemu.

Wymagania sprzętowe i programowe

System wymaga środowiska Windows 10/11 z aktywnym podsystemem WSL2 (Ubuntu) dla procesu treningu. Do inferencji wystarczy samo środowisko Windows. Wymagane jest GPU NVIDIA z minimum 8 GB VRAM (dla treningu w trybie 4-bit kwantyzacji). Zależności programowe obejmują: serwer Ollama (inferencja modelu), Python 3.11+, Node.js 18+ oraz Git.

Proces instalacji — skrypt automatyczny (setup.cmd)

W projekcie zaimplementowano skrypt setup.cmd, który automatyzuje cały proces konfiguracji środowiska. Skrypt stosuje wzorzec ``goto :done`` zamiast natychmiastowego przerywania — w przypadku błędu wyświetla komunikat i czeka na potwierdzenie użytkownika, pozwalając na diagnostykę problemu. Skrypt wykonuje następujące kroki w ściśle określonej kolejności:

1. Weryfikacja obecności wymaganych programów (Python 3.11+, npm, Ollama) na zmiennej PATH. Dla każdego znalezionej programu wyświetlany jest komunikat [OK]; w przypadku braku któregoś z nich, skrypt wyświetla instrukcję instalacji wraz z linkiem do pobrania i przerywa dalsze kroki.
2. Inicjalizacja submodułów Git (repozytorium llama.cpp) za pomocą `git submodule update --init --recursive`.
3. Utworzenie wirtualnego środowiska Python (venv) w katalogu backend/.venv i instalacja zależności z pliku requirements.txt za pomocą pip.
4. Instalacja zależności frontendowych (npm install) w katalogu frontend/.
5. Konfiguracja środowiska treningowego WSL2 — skrypt sprawdza dostępność podsystemu WSL2, weryfikuje obecność python3 i modułu python3-venv, a następnie tworzy wirtualne środowisko (.venv-wsl) i instaluje zależności treningowe z pliku backend/requirements-wsl.txt. W przypadku braku WSL2 lub niepełnych wymagań, wyświetlany jest komunikat z instrukcją instalacji, a krok jest pomijany bez przerywania konfiguracji pozostałych komponentów.
6. Aktualizacja ścieżki ADAPTER w pliku Modelfile na ścieżkę bezwzględną (wymaganie serwera Ollama) i rejestracja modelu bielik-lora-mipd za pomocą polecenia ollama create. Jeśli model jest już zarejestrowany w Ollama, krok jest pomijany.

Katalog model/ (~7.7 GB) nie jest wersjonowany w repozytorium i musi zostać pobrany oddzielnie — pliki modelu są dostępne pod adresem <https://drive.google.com/file/d/1b17N4rKeinj1ahqTz-nhMtxZT8k02OYD/view?usp=sharing>. Skrypt setup.cmd wykrywa brak pliku adaptera i wyświetla odpowiedni komunikat ostrzegawczy, nie przerywając konfiguracji pozostałych komponentów.

Skrypt setup.cmd wymaga dostępu do serwera Ollama podczas rejestracji modelu (krok 6). Skrypt run_app.cmd automatycznie uruchamia serwer Ollama, jeśli nie jest on dostępny. Idempotentność obu skryptów gwarantuje bezpieczeństwo ponownego uruchomienia: istniejące środowisko wirtualne, zainstalowane pakiety npm, skonfigurowane środowisko WSL2 i zarejestrowany model nie są nadpisywane. Dodatkowo, w przypadku błędu tworzenia środowiska wirtualnego Python, skrypt wyświetla szczegółowe instrukcje rozwiązywania problemów (np. ponowna instalacja Pythona z opcją „Add Python to PATH”).

Konfiguracja środowiska treningowego (WSL2)

Proces douczania modelu (Supervised Fine-Tuning) wymaga środowiska Linux i jest realizowany w podsystemie WSL2. Skrypt setup.cmd automatycznie wykrywa dostępność WSL2, weryfikuje wymagania (python3, python3-venv) i tworzy odizolowane środowisko wirtualne (.venv-wsl) z zależnościami z pliku requirements-wsl.txt. W przypadku niepowodzenia, skrypt wyświetla instrukcję ręcznej instalacji (`sudo apt install python3`

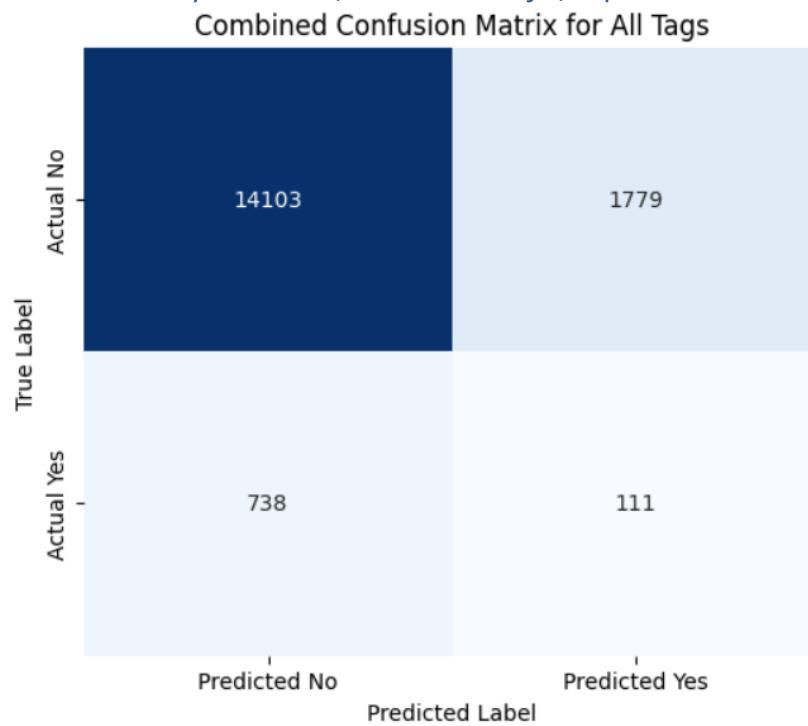
python3-venv python3-pip). Ręczna instalacja zależności w terminalu Ubuntu: `pip install unsloth bitsandbytes accelerate torch trl datasets gguf`. Trening nie jest uruchamiany bezpośrednio z wiersza poleceń — jest inicjowany z poziomu interfejsu użytkownika (Panel Ekspercki). Backend na Windowsie uruchamia proces treningowy w WSL za pomocą polecenia `wsl --exec python3 -u -m app.training.trainer`, przekazując ścieżki do danych i modelu jako argumenty. Postęp treningu jest raportowany w czasie rzeczywistym do backendu przez wywołania HTTP POST na endpoint `/training/progress`. Do uruchomienia wszystkich komponentów systemu służy skrypt `run_app.cmd`, który w osobnych oknach terminala uruchamia usługi w następującej kolejności:

Uruchomienie systemu — kolejność startu usług

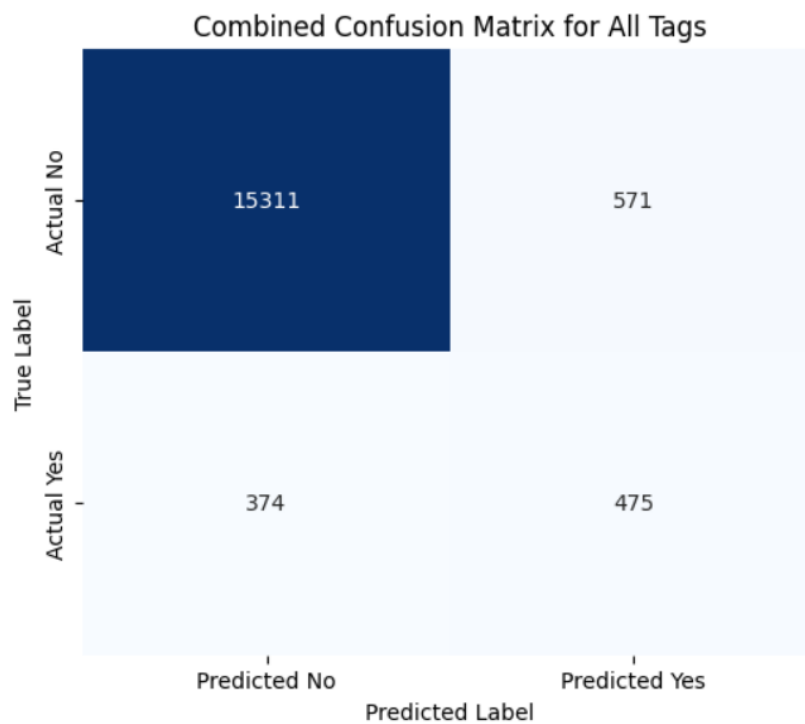
1. Serwer Ollama (port 11434) — musi być uruchomiony jako pierwszy, ponieważ backend i proces treningu zależą od jego dostępności. Odpowiada za inferencję modelu językowego.
2. Backend FastAPI (port 8000) — serwis orkiestracyjny. Uruchamiany poleceniem `python -m app.main` z aktywowanym środowiskiem wirtualnym. Obejmuje endpointy REST (`/analize`, `/upload`, `/train`, `/promote`), WebSocket (`/ws/training/status`) oraz komunikację z bazą SQLite i serwerem Ollama.
3. Frontend Vite/React (port 5173) — interfejs użytkownika. Uruchamiany poleceniem `npm run dev`. Łączy się z backendem za pomocą REST API i WebSocket, udostępniając panel analizy tekstu oraz Panel Ekspercki.

Po uruchomieniu system jest dostępny pod adresem `http://localhost:5173`. Reset do stanu początkowego (przywrócenie adaptera xai-adapter jako aktywny model, usunięcie raportu benchmark i artefaktów treningowych) realizowany jest przez skrypt `reset_state.cmd`.

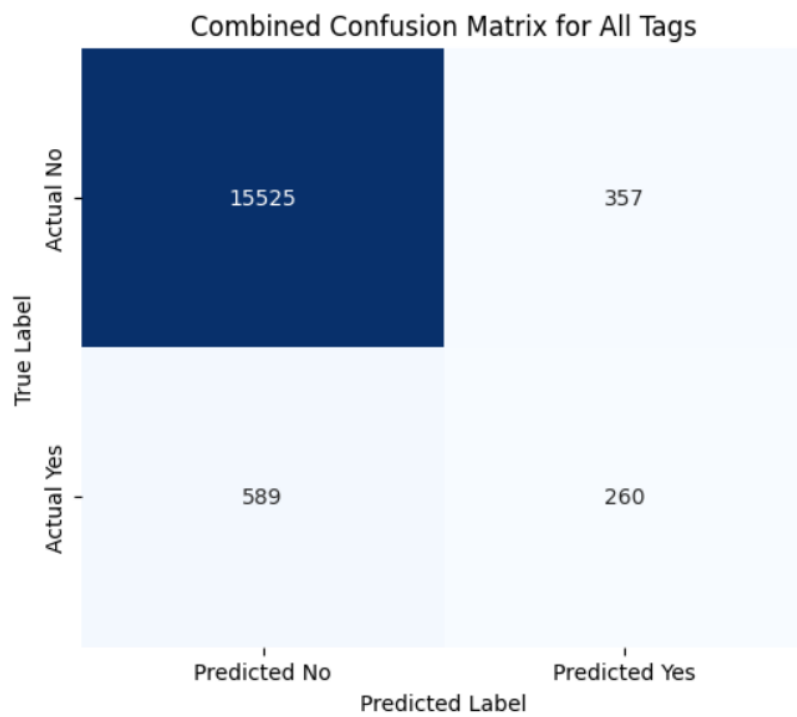
3. Zrzuty ekranu, wizualizacje, itp.



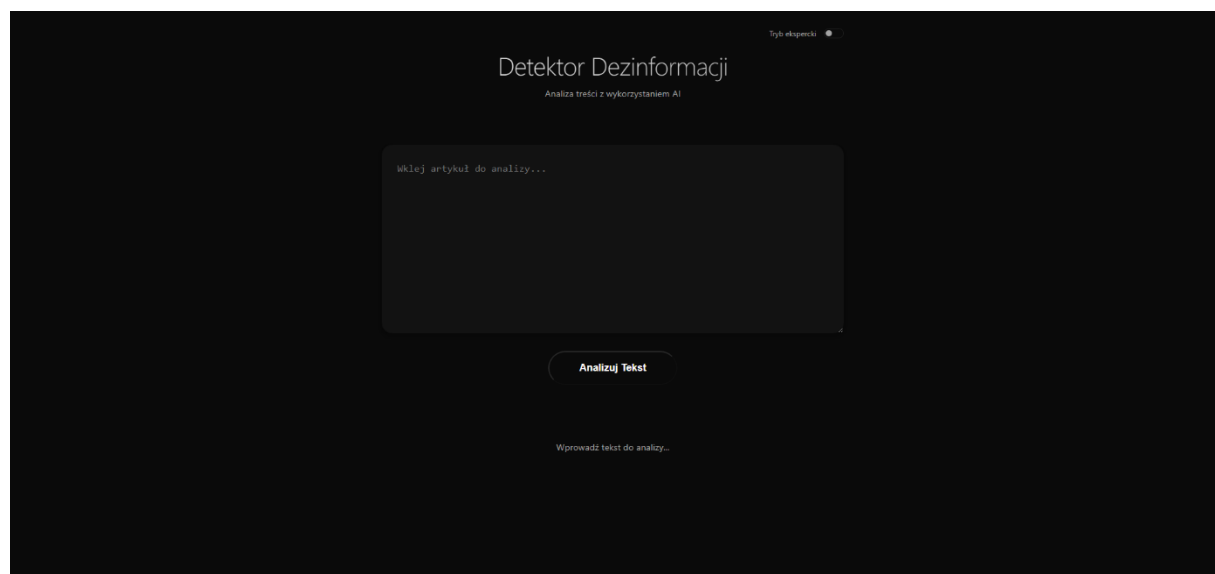
No adapter confusion matrix



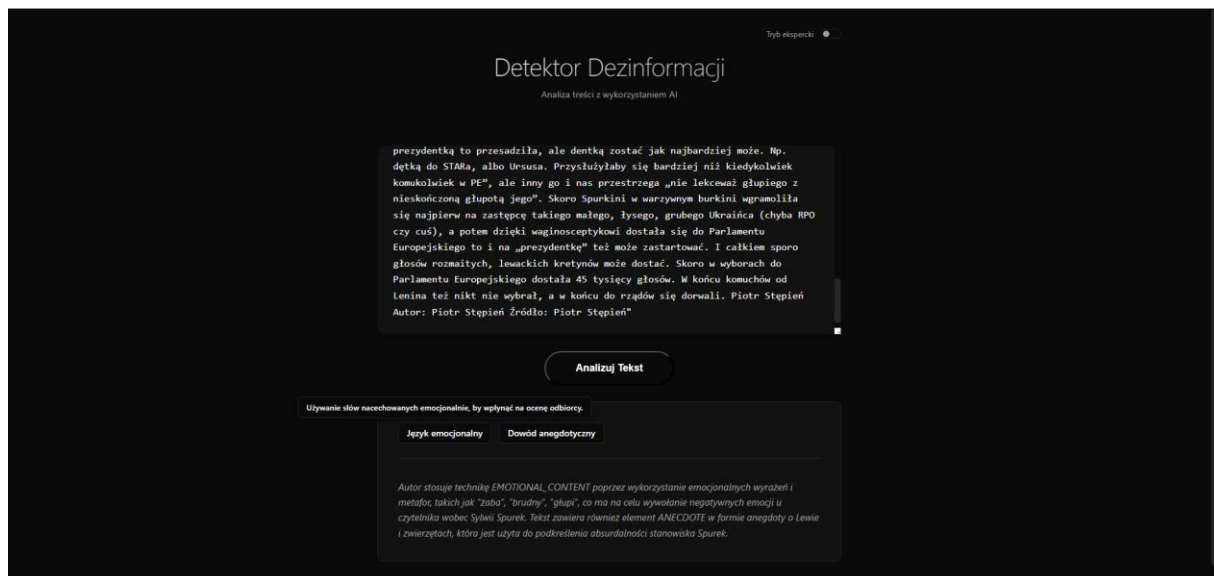
Prototype adapter confusion matrix



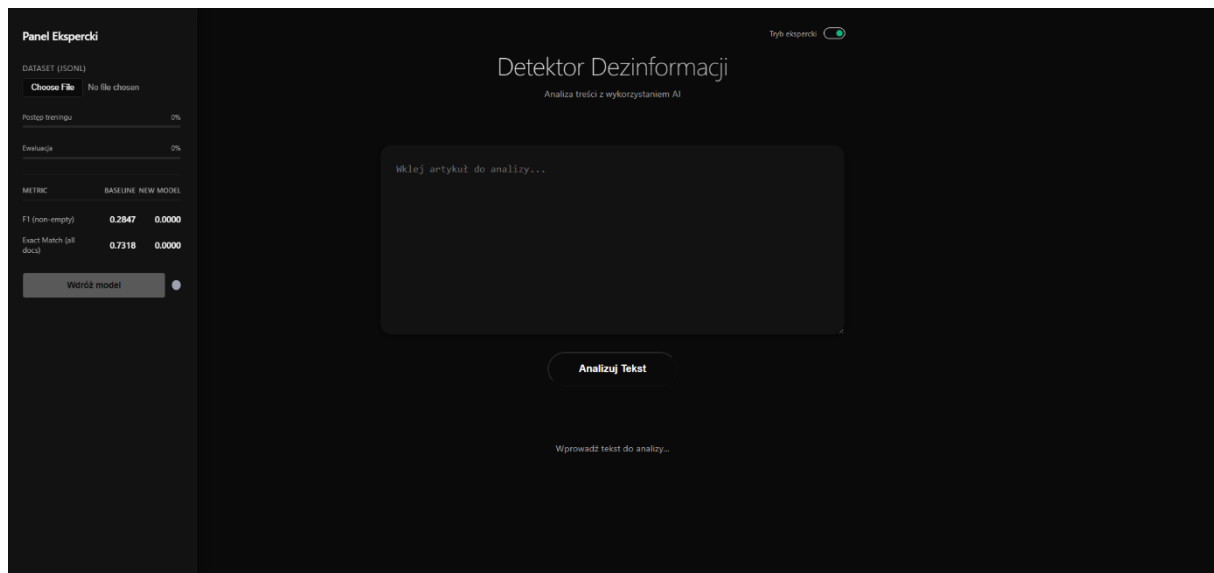
Xai adapter confusion matrix



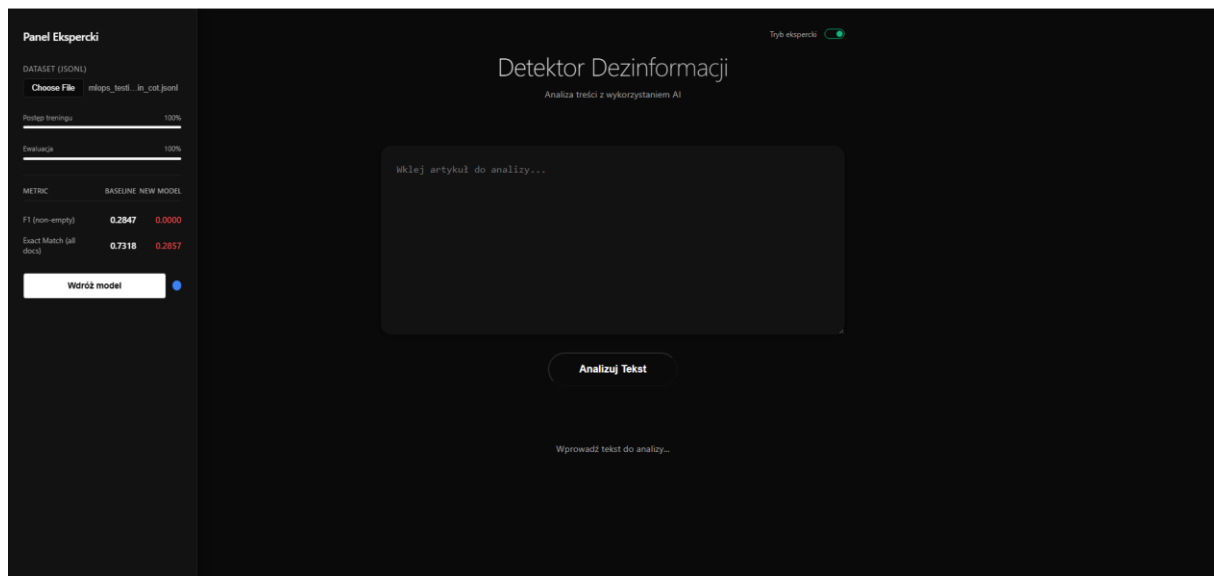
Interfejs użytkownika



Wynik wnioskowania: interfejs wyświetla tagi wraz z odpowiedziami i objaśnieniami.



Interfejs panelu eksperckiego



Interfejs wdrożenia.

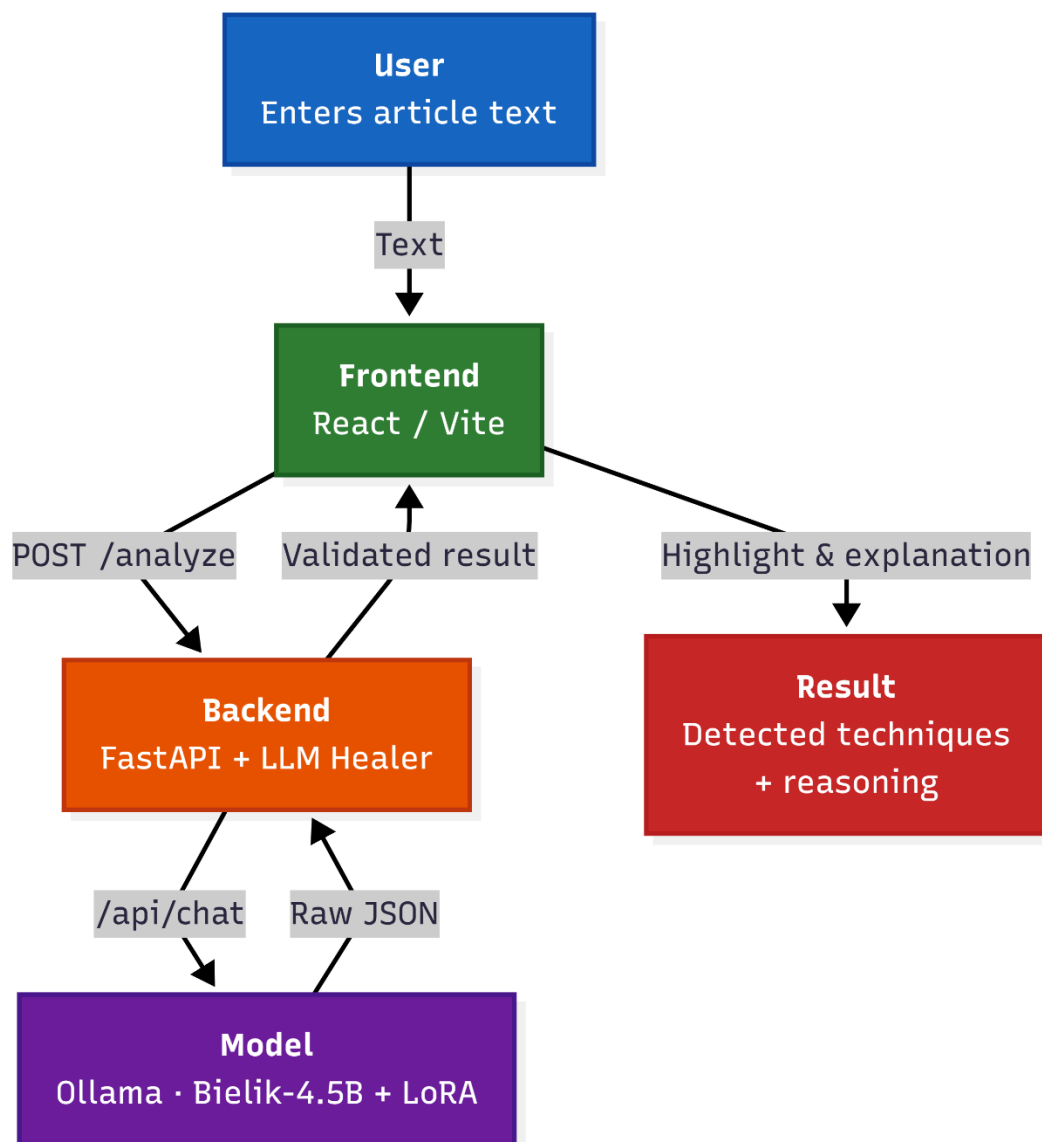


Diagram architektury wnioskowania

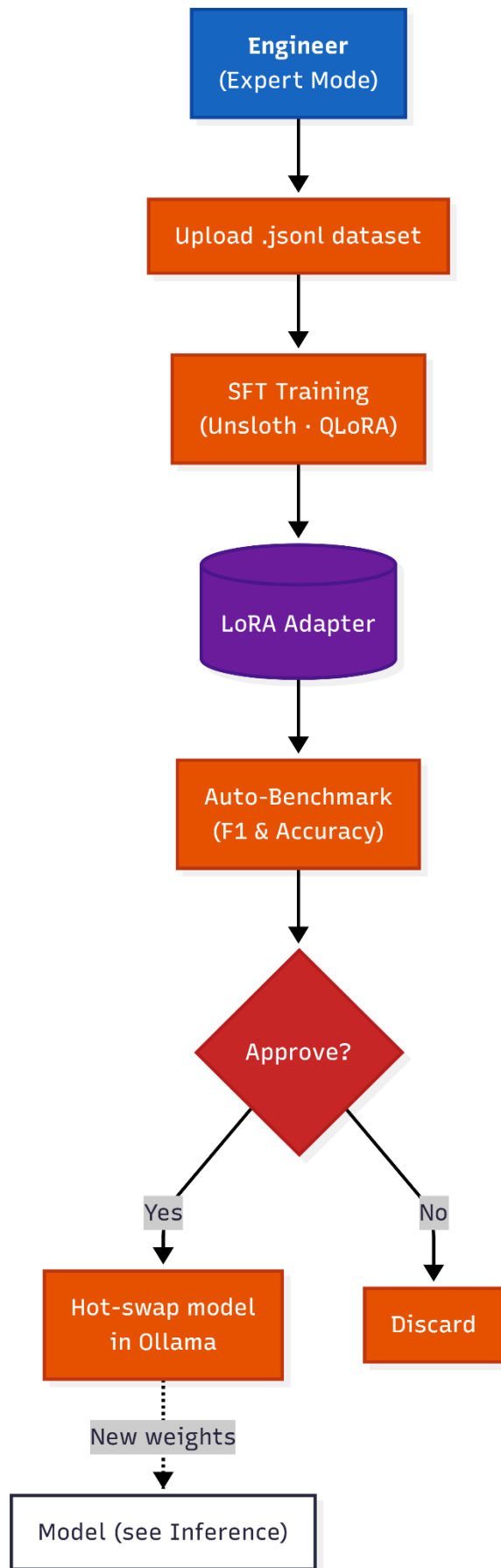


Diagram architektury MLOps

4. Wnioski i perspektywy rozwoju

Główny cel pracy, zdefiniowany jako zaprojektowanie i implementacja kompletnego systemu informatycznego do detekcji manipulacji w tekstach polskojęzycznych, został zrealizowany. Powstał w pełni funkcjonalny artefakt inżynierski, który integruje zaawansowane modele językowe (LLM) z nowoczesną architekturą aplikacji internetowej.

Udało się skutecznie rozwiązać kluczowe wyzwania techniczne:

- * Zaadaptowano model Bielik-4.5B do specyficznego zadania detekcji błędów logicznych poprzez proces nadzorowanego dostrajania (SFT) z wykorzystaniem techniki QLoRA.
- * Zrozumiano i zaimplementowano proces destylacji wiedzy w celu wygenerowania wyjaśnień w języku naturalnym (NLE), co nadało systemowi cechę wyjaśnialności (XAI).
- * Zbudowano kompletną pętlę MLOps, umożliwiającą automatyczne douczanie i wdrażanie modelu w trybie ciągłym (z udziałem człowieka), co wykracza poza standardowy zakres prac inżynierskich.

System działa lokalnie, wykorzystując optymalizacje (kwantyzacja 4-bitowa, dynamiczne ładowanie adapterów), co czyni go dostępnym bez konieczności inwestowania w kosztowną infrastrukturę chmurową. Osiągnięto kompromis między jakością detekcji a wymaganiami sprzętowymi, dostarczając narzędzie gotowe do dalszego rozwoju.

Perspektywy rozwoju

Zidentyfikowane w toku prac ograniczenia projektu wyznaczają bezpośrednie kierunki jego dalszego rozwoju:

1. Rozbudowa infrastruktury i skalowanie: Obecna wersja systemu, będąca weryfikacją koncepcji (Proof-of-Concept), jest ograniczona zasobami sprzętu konsumenckiego. Naturalnym krokiem jest migracja rozwiązania na środowisko serwerowe z profesjonalnymi układami GPU (VRAM > 24GB). Pozwoliłoby to na pełne wykorzystanie okna kontekstowego modelu (do 32k tokenów) bez kompromisów wydajnościowych oraz obsługę wielu żądań jednocześnie.
2. Automatyczna Walidacja Wyjaśnień (model jako sędzia – LLM-as-a-Judge): Ze względu na skalę zbioru danych, ręczna ocena tysięcy wyjaśnień przez ekspertów jest nieefektywna kosztowo i czasowo. Perspektywicznym kierunkiem jest wdrożenie metodologii "model jako sędzia" (LLM-as-a-Judge), w której potężny model językowy (np. GPT-4) ocenia spójność i poprawność logiczną generowanych przez system wyjaśnień (NLE). Rola ludzkiego eksperta ograniczałaby się wówczas do weryfikacji reprezentatywnej, losowej próbki ocen modelowych, co pozwoliłoby na skalowalną walidację jakości modułu XAI.
3. Zbalansowanie i rozszerzenie zbioru danych: Aby zredukować "konserwatywność" modelu (tendencję do nieoznaczania technik w przypadkach niejednoznacznych), należy wzbogacić zbiór treningowy o większą liczbę przykładów pozytywnych (zawierających manipulację), redukując relatywną nadreprezentację próbek pustych. Możliwe jest również rozszerzenie taksonomii wykrywanych błędów o nowe kategorie.

4. Wdrożenie zaawansowanych mechanizmów bezpieczeństwa: Ewentualne wdrożenie produkcyjne wymaga zastąpienia obecnego, uproszczonego modułu logowania pełnym systemem uwierzytelniania i autoryzacji opartym na rolach (RBAC). Pozwoli to na bezpieczne audytowanie zmian wprowadzanych przez ekspertów do zbioru treningowego "Złotych Próbek".

5. Łagodzenie stronniczości (Bias Mitigation): Istotnym kierunkiem badawczym jest analiza i eliminacja potencjalnych uprzedzeń, które model mógł odziedziczyć w procesie destylacji wiedzy od modelu "Nauczyciela". Opracowanie metod filtracji danych treningowych pod kątem neutralności światopoglądowej zwiększy obiektywność systemu.

5. Bibliografia/źródła

1. Hsieh, C.-Y., Li, C.-L., Yeh, C.-K., Nakhost, H., Fujii, Y., Ratner, A., Krishna, R., Lee, C.-Y., & Pfister, T. (2023). *Distilling Step-by-Step! Outperforming Larger Language Models with Less Training Data and Smaller Model Sizes*. Findings of the Association for Computational Linguistics: ACL 2023. <https://doi.org/10.18653/v1/2023.findings-acl.507>
2. Lei, T., Barzilay, R., & Jaakkola, T. (2016). *Rationalizing Neural Predictions*. arXiv preprint arXiv:1606.04155. <https://doi.org/10.48550/arXiv.1606.04155>
3. Camburu, O.-M., Rocktäschel, T., Lukasiewicz, T., & Blunsom, P. (2018). *e-SNLI: Natural Language Inference with Natural Language Explanations*. arXiv preprint arXiv:1812.01193. <https://doi.org/10.48550/arXiv.1812.01193>
4. Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2021). *LoRA: Low-Rank Adaptation of Large Language Models*. arXiv preprint arXiv:2106.09685. <https://doi.org/10.48550/arXiv.2106.09685>
5. Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). *QLoRA: Efficient Finetuning of Quantized LLMs*. arXiv preprint arXiv:2305.14314. <https://doi.org/10.48550/arXiv.2305.14314>
6. Han, D., & Liu, C. (2023). *Unsloth: An Open-Source Library for Faster LLM Fine-Tuning*. GitHub. <https://github.com/unslothai/unsloth>
7. Liu, X., Yan, H., Zhang, S., An, C., Qiu, X., & Lin, D. (2023). *Scaling Laws of RoPE-based Extrapolation*. arXiv preprint arXiv:2310.05209. <https://doi.org/10.48550/arXiv.2310.05209>
8. Modzelewski, A., Da San Martino, G., Savov, P., Wilczyńska, M. A., & Wierzbicki, A. (2024). *MIPD: Exploring Manipulation and Intention In a Novel Corpus of Polish Disinformation*. Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing.
9. SpeakLeash Team (Ociepa, K. et al.). (2024). *Bielik-4.5B-v3: Polish Large Language Model*. Technical Report. <https://huggingface.co/speakleash/Bielik-4.5B-v3>
10. Yang, A., Yang, B., Zhang, B., et al. (2024). *Qwen2.5 Technical Report*. arXiv preprint arXiv:2412.15115. <https://doi.org/10.48550/arXiv.2412.15115>
11. Gerganov, G. (2023). *llama.cpp: Inference of LLaMA model in pure C/C++*. GitHub. <https://github.com/ggerganov/llama.cpp>